Modules and Packages.

PAGE NO.....
DATE / /

1. What is a Module?

→ A module is a file containing Python code (e.g. functions, classes, variables). It allows you to organize your code into reusable components.

Example: Module

Suppose you have a file named math_operation.py

```
Python
def add(a, b):
    return a+b

def subtract(a, b)
    return a-b
```

This file is a module named math_operations

2. Importing Modules.

~~python~~ Syntax

```
import module_name
```

E.g

```
import math_operations
result = math_operations.add(10,5)
print(result) # output: 15
```

Here,

- The `math_operations` module is imported.
- You can access its functions using `module_name.function_name`.

PAGE NO.....
DATE / /

3) Importing specific items.

Syntax

from module_name import item_name

E.g.

```
# main.py
from math_operations import add

result = add(10,5)
print(result) # output: 15.
```

4) Importing All items

Syntax

```
from module_name import *
```

E.g. Importing All items

```
# main.py
from math_operations import *

result1 = add(10,5)
result2 = subtract(10,5).
print(result1, result2) # output: 15.5.
```

PAGE NO.....
DATE / /

5) What is Package?

~~my-pac~~

→ A package is a collection of modules organised in a directory. It allows you to group related modules together.

Structure of a Package.

```
my_package
    --init--.py
    module1.py
    module2.py
    subpackage/
        --init--.py
        module3.py
```

6) Importing from Packages.

```
from my_package import module1
from my_package.subpackage import module3

module1.function1()
module3.function3()
```

(8) Relative imports

```
from .import module2
from .subpackage import module3
```

Here.

* . refers to the current package (my_package)
* .subpackage refers to the subpackage inside the current package

PAGE NO.....
DATE / /

(9) The __init__.py file

→ The __init__.py file is used to initialize a package. It marks a directory as a package. It can be:

1) Empty: Just to indicate that the directory is a package.

(2) Non-empty: Contains initialization code or defines what gets imported when you use from package import *

Why is __init__.py important?

1) Marks a Directory as a Package.
(2) Initialization Code
(3) Controls what gets Imported.

How to Use __init__.py?

1. Empty __init__.py

Directory Structure:

```
my_package/
    __init__.py
    module1.py
    module2.py
```

PAGE NO.....
DATE / /

python (.Empty)

```
# my-package/__init__.py
# This file is empty.

# main
from my-package import module1, module2

module1.function()
module2.function()
```

(2) Non-empty

```
# my-package/__init__.py
print("Initializing my-package ...")

# Import specific function from modules.
from module1 import function1.
from module2 import function2

# main.py
import my-package  # output: Initializing
                   my-package.
my-package.function1().
my-package.function2()
```

(3) __init__.py with __all__

```
# my-package/__init__.py
from . module1 import function1
from . module2 import function2

__all__ = ['function1', 'function2']
# only these will be imported with 'from package
  import *'
```

PAGE NO.....
DATE / /

```
# main.py
from my_package import *

function1()      # works
function2()      # works.

# module1 and module2 are not imported.
# module1.function() # This will raise an error.
```

(10) Installing Third-Party Packages

```
pip install requests

# using 3rd-party Package.
import requests.
response = requests.get("https://www.example.com")
print(response.status_code).
```